

# Lists & Dictionaries

Python's Most Essential Data Structures

## ■ LEARNING OUTCOME

By the end of this module you will work fluently with Python lists, dictionaries, tuples, and sets -- using all their methods, slicing, sorting, and the collections module for advanced use cases.

### ■ Skills You Will Gain

- Master all list methods: append, extend, insert, sort, pop
- Use list slicing for powerful data manipulation
- Work with nested lists and dicts (JSON-like structures)
- Use all dict methods: get, setdefault, update, pop
- Use collections: Counter, defaultdict, OrderedDict, namedtuple
- Understand set operations: union, intersection, difference

■ Lists and dictionaries are used in literally every Python program. Mastering them -- all methods, slicing, sorting, comprehensions -- unlocks 80% of Python data manipulation power.

## SECTION 1

# Lists -- Complete Reference

```
# Creating lists empty = [] nums = [1, 2, 3, 4, 5] mixed = [1, "hello", True, None, [1,2]]
range_list = list(range(1, 11)) # [1..10] zeros = [0] * 5 # [0,0,0,0,0] copy = nums[:] # shallow
copy via slice # ACCESSING print(nums[0]) # 1 first print(nums[-1]) # 5 last print(nums[-2]) # 4
second to last # SLICING [start:stop:step] print(nums[1:4]) # [2,3,4] indexes 1,2,3 print(nums[:3])
# [1,2,3] first 3 print(nums[2:]) # [3,4,5] from index 2 print(nums[::2]) # [1,3,5] every 2nd
print(nums[::-1]) # [5,4,3,2,1] reversed # MUTATING METHODS nums.append(6) # add single item to
end: [1,2,3,4,5,6] nums.extend([7,8,9]) # add multiple items: [1..9] nums.insert(0, 0) # insert at
index 0 nums.pop() # remove and return last: 9 nums.pop(0) # remove and return at index 0
nums.remove(3) # remove first occurrence of value 3 nums.clear() # empty the list nums.reverse() #
reverse in-place nums.sort() # sort ascending in-place nums.sort(reverse=True) # sort descending
in-place nums.sort(key=lambda x: abs(x)) # custom sort key # NON-MUTATING print(nums.index(3)) #
index of first 3 print(nums.count(3)) # count occurrences of 3 new_list = sorted(nums) # return new
sorted list rev_list = list(reversed(nums)) # return new reversed # USEFUL OPERATIONS
print(len(nums)) # length print(sum(nums)) # sum print(min(nums)) # minimum print(max(nums)) #
maximum print(3 in nums) # membership test print(nums + [10,11]) # concatenate combined = [*nums,
*other] # unpack and merge
```

## SECTION 2

# Dictionaries -- Complete Reference

```
# Creating dicts empty = {} person = {"name":"Alice","age":28,"city":"Mumbai"} from_keys =
dict.fromkeys(["a","b","c"], 0) # {a:0,b:0,c:0} comprehension = {k:v for k,v in pairs} # ACCESSING
print(person["name"]) # "Alice" (KeyError if missing) print(person.get("name")) # "Alice" (None if
missing) print(person.get("phone","N/A")) # "N/A" (default if missing) # MUTATING person["email"] =
"alice@example.com" # add/update key person.update({"age":29,"job":"dev"}) # update multiple
deleted = person.pop("email") # remove and return person.pop("phone", None) # safe delete (no error
if missing) del person["city"] # delete key person.clear() # empty dict # QUERYING
print(person.keys()) # dict_keys -- view object print(person.values()) # dict_values -- view object
print(person.items()) # dict_items -- (key,value) tuples print("name" in person) # True --
membership on keys print(len(person)) # number of key-value pairs #.setdefault -- get if exists,
set default if not person.setdefault("role","viewer") # only sets if key absent # Merge dicts
(Python 3.9+) merged = base_config | user_config # new dict base_config |= user_config # in-place
update # Iterate for key in person: # iterate keys print(key) for key, val in person.items(): #
iterate key-value pairs print(f"{key}: {val}") # Nested dict users = { "alice":
{"age":28,"scores":[90,85,92]}, "bob": {"age":35,"scores":[78,88,95]} }
print(users["alice"]["scores"][0]) # 90 print(users.get("carol",{}).get("age","Unknown")) # safe
nested access
```

## SECTION 3

# Tuples, Sets and Collections Module

```
# TUPLES -- immutable ordered sequence point = (3, 4) rgb = (255, 128, 0) singleton = (42,) # MUST
have trailing comma for single-element packed = 1, 2, 3 # parentheses optional a, b, c = packed #
unpacking x, *rest = (1,2,3,4) # starred unpacking # Tuple uses: multiple return values, dict keys,
namedtuple, data integrity # SETS -- unordered unique items s1 = {1,2,3,4,5} s2 = {3,4,5,6,7} s3 =
set([1,2,2,3]) # {1,2,3} -- dedup from list print(s1 | s2) # {1,2,3,4,5,6,7} union print(s1 & s2) #
{3,4,5} intersection print(s1 - s2) # {1,2} difference print(s1 ^ s2) # {1,2,6,7} symmetric
difference s1.add(6) # add single item s1.discard(10) # safe remove (no error if absent)
print(s1.issubset(s2)) # s1 subset of s2? print(s1.issuperset(s2)) # s1 superset of s2? # Remove
duplicates from list: unique = list(set(my_list)) # collections module from collections import
Counter, defaultdict, OrderedDict, namedtuple, deque # Counter -- count occurrences words =
["apple","banana","apple","cherry","banana","apple"] c = Counter(words) print(c) #
```

```
Counter({"apple":3,"banana":2,"cherry":1}) print(c.most_common(2)) # [('apple',3),('banana',2)]
print(c["apple"]) # 3 # defaultdict -- auto-create missing keys grouped = defaultdict(list) for
word in words: grouped[word[0]].append(word) # group by first letter # namedtuple -- tuple with
named fields Point = namedtuple("Point",["x","y"]) Person =
namedtuple("Person",["name","age","city"]) p = Point(3, 4) alice = Person("Alice",28,"Mumbai")
print(alice.name) # Alice print(alice._asdict()) # OrderedDict # deque -- O(1) append/pop from both
ends q = deque([1,2,3]) q.appendleft(0) # fast left append q.popleft() # fast left pop
```

## ■ PRACTICE Q & A

**Q1.** What is the difference between `list.append()` and `list.extend()`?

*A: `append()` adds ONE item to the end -- always increases length by 1. `extend()` adds all items from an iterable -- increases length by `len(iterable)`. `append([1,2])` adds the list as one item; `extend([1,2])` adds 1 and 2 separately.*

**Q2.** What is the difference between a list and a tuple?

*A: List: mutable (can add/remove/change items), uses `[]`, slightly slower. Tuple: immutable (cannot change after creation), uses `()`, slightly faster, can be used as dict keys or set members.*

**Q3.** When would you use a `defaultdict`?

*A: When you need to group or count items and don't want to check if a key exists first. `defaultdict(list)` auto-creates an empty list for new keys. `defaultdict(int)` auto-creates 0 for new integer keys.*

**Q4.** How do you safely access a nested dict without `KeyError`?

*A: Use `.get()` at each level: `d.get('user',{}).get('address',{}).get('city','Unknown')`. Or use Python 3.10+ `match/case`, or write a helper function with `try/except`.*

**Q5.** What is the time complexity of `'in'` for list vs set?

*A: List `'in'`:  $O(n)$  -- scans every element. Set `'in'`:  $O(1)$  average -- uses hash lookup. For frequent membership tests on large collections, always use a set.*

| ■ Lists & Dicts Cheat Sheet                                            |                                   |
|------------------------------------------------------------------------|-----------------------------------|
| <code>lst.append(x)</code>                                             | Add single item to end            |
| <code>lst.extend([x,y,z])</code>                                       | Add multiple items                |
| <code>lst.insert(i,x)</code>                                           | Insert at index                   |
| <code>lst.pop()</code> / <code>lst.pop(i)</code>                       | Remove and return last / at index |
| <code>lst.sort(key=fn)</code>                                          | Sort in-place with key            |
| <code>sorted(lst,key=fn,reverse=True)</code>                           | Return new sorted list            |
| <code>lst[1:4]</code> / <code>lst[::-1]</code>                         | Slice / reverse                   |
| <code>d.get(key,default)</code>                                        | Safe dict access                  |
| <code>d.setdefault(k,default)</code>                                   | Set only if key missing           |
| <code>d.items()</code> / <code>.keys()</code> / <code>.values()</code> | Dict views                        |
| <code>Counter(iterable)</code>                                         | Count element frequencies         |
| <code>defaultdict(list)</code>                                         | Auto-create list for new keys     |